



Game Programming

Assignment #03

Name: Muhammad Shaaf Raheel

Sap: 60115

Section: CS-5A



Game Choosed: among us

1. Game Overview

Among us is a multiplayer online game for 4 to 15 players. The game takes place on a spaceship, a headquarters, or a planet base.

Core Gameplay Mechanics:

Roles: Players are secretly assigned as either Crewmates or Impostors.

Crewmate Goal: Complete all given "tasks" (like fixing wires or swiping a card) or identify and vote out all Impostors.

Impostor Goal: Kill Crewmates or cause sabotages (like cutting the lights) without being caught. The Impostor blends in as a normal player.

Key Actions:

Tasks: Small mini-games that Crewmates must finish.

Sabotage: Impostors can cause emergencies that Crewmates must fix.

Kill: Impostors can kill nearby Crewmates, but with a cool down timer.

Report: Any player can report a dead body, starting a meeting.

Meeting/Voting: When a body is reported or an emergency meeting is called, all players discuss and vote to eject a suspected Impostor.

2. High-Level Architecture

To build Among Us, we need several major systems working together. Here are the main systems and what they do.

Player System

Manages each player's data: their role (Crew/Impostor), color, name, position, and status (alive/dead).

Network System

Syncs game data between all players and the server. Handles joining, leaving, and lag.

Matchmaking System

Groups players into a lobby. Let the host choose the map and game settings (e.g., kill cool down, number of Impostors).

Task System

Spawns tasks on the map for Crewmates. Tracks which tasks are complete. Sends "Task Complete" updates to the server.

Sabotage & Fix System

Allows Impostors to trigger disasters (Oxygen, Reactor, Lights). Allows all players to fix them by going to a location and doing a mini-game.

Combat/Kill System

Handles the Impostor's kill action. Checks cool down, distance, and line of sight. Updates a player's status to "dead" and spawns a body.

Meeting & Voting System

Starts a meeting (on report or button press). Collects votes from alive players. Ejects the player with the most votes.

Game State Manager

Tracks the current phase of the game: Lobby, Task Phase, Meeting Phase, game over. This is the brain of the game.

UI System

Shows the player map, task list, kill button (for Impostors), sabotage menu, meeting chat, and voting screen.

3. Code Structure

Main Classes and Their Responsibilities:

GameManager (Central class)

Responsibility: Runs the Game State Manager. Decides when to switch phases (e.g., "Task phase is over, start meeting").

Communication: Tells all other systems when the phase changes.

Player

Responsibility: Stores data for one player: playerID, role (enum: Crew, Impostor), isAlive, position, color.

Communication: Listens to the Kill System (to become dead) and Meeting System (to cast a vote).

NetworkManager

Responsibility: Sends and receives messages. Uses a server-authoritative model (the server has the final say).

Communication: It receives actions (like "I killed player X") and checks if they are valid before telling all clients.

TaskManager

Responsibility: Holds a list of all tasks on the current map. For each player, tracks their personal task progress.

Communication: When a Crewmate finishes a task, TaskManager tells GameManager. If all tasks are done, GameManager ends the game with a Crewmate win.

SabotageManager

Responsibility: Handles active sabotages. Has a timer for critical sabotages (Oxygen/Reactor).

Communication: An Impostor calls StartSabotage(type). This tells the UI to show an alarm. If the timer hits zero, it tells GameManager that Impostors win.

MeetingManager

Responsibility: Runs the voting process. Collects votes, calculates the result, and ejects a player.

Communication: After voting, it tells GameManager to go back to Task Phase. If a player is ejected, it tells Player to mark them as dead.

4. Design Decisions

I chose this architecture for Among Us for several specific reasons.

Decision 1: Server-Authoritative Architecture

What it means: The central server is the absolute source of truth. A player's game can suggest an action (like "I want to kill"), but the server approves or denies it.

Why it's best: This prevents cheating. If a player hacks their local game to remove the kill cooldown, the server would still check the real cooldown and say "No." This is standard for all competitive multiplayer games.

Decision 2: Component-Based Systems (TaskManager, SabotageManager)

What it means: Instead of one giant class doing everything, we split work into separate, focused managers.

Why it's best: It makes the code easy to change. If the developers want to add a new type of sabotage (e.g., "Melt the Ice"), they only change the SabotageManager class. They don't touch the TaskManager or Player class. This reduces bugs.

Decision 3: Clear Game State Machine (GameManager)

What it means: The game has clear phases (Lobby -> Task -> Meeting -> Game Over), and the GameManager controls the switch between them.

Why it's best: Among Us is a turn-based game inside a real-time world. The Meeting phase works like a turn. By having a state machine, the game always knows what actions are allowed. For example, the "Kill" button is only active during the Task Phase. This logic is simple and safe.

Decision 4: Separate Network Logic from Game Logic

What it means: The NetworkManager only worries about sending data. It does not know how a task works. The TaskManager does not know how to send data to another player.

Why it's best: This makes the game easier to test. A developer can test the TaskManager on a single computer without the internet. It also makes it easier to switch network libraries later if needed.



My thoughts:

This architecture is simple, secure, and scalable. It separates concerns (each manager does one job), prevents cheating with a server-authoritative model, and makes it easy to add new features to Among Us, like new maps, tasks, or sabotages.